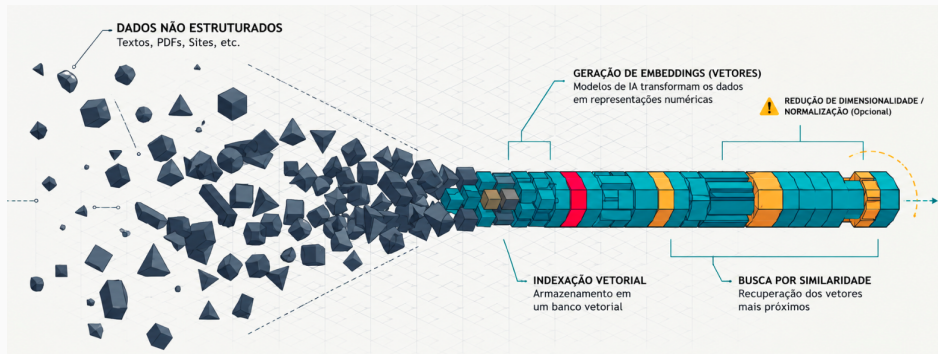




Recuperação Vetorial e Contexto - Banco de Dados

Embeddings, Similaridade e Limitações

Prof^a Dra Simone Dominico

Especialização em Inteligência Artificial Generativa Aplicada

Problema e contexto

Objetivos da Aula

- Distinguir dado, conhecimento e contexto
- Compreender recuperação como aproximação
- Entender embeddings e espaço vetorial
- Analisar trade-offs de recuperação
- Implementar pipeline simples
- Diagnosticar falhas de recuperação
- Analisar ruído e redundância

Problema Inicial

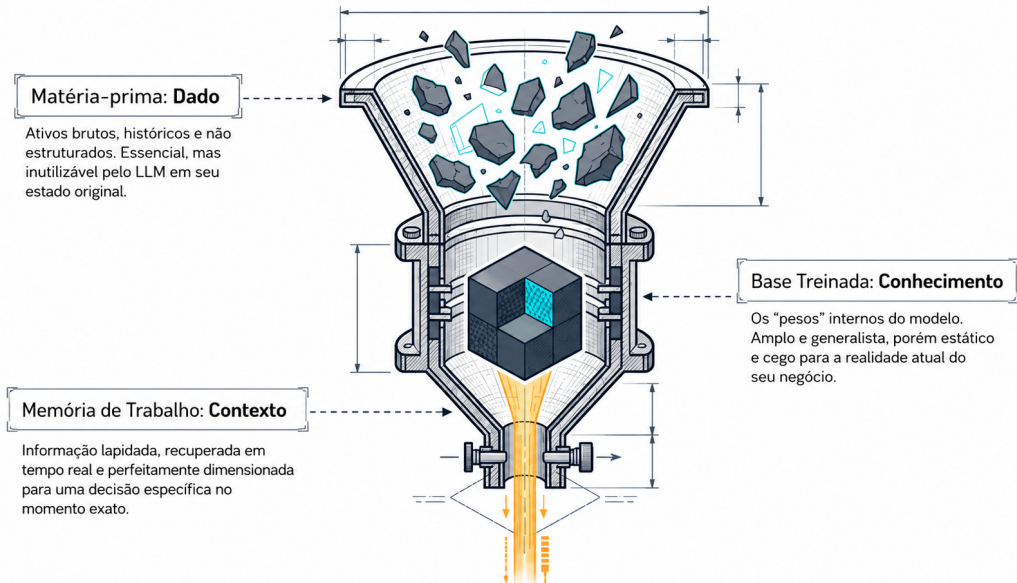
Perguntas:

- O curso possui estágio obrigatório?
- Qual é a frequência mínima?
- A disciplina aborda SQL?
- O curso possui TCC?

Pergunta central:

Como um sistema consegue responder isso usando documentos?

Dado, Conhecimento e Contexto



Dado

- informação isolada;
- matéria-prima bruta.

Exemplo

“220 horas”
registros, documentos...

Dado, Conhecimento e Contexto

Dado

- informação isolada;
- matéria-prima bruta.

Exemplo

“220 horas”
registros, documentos...

Conhecimento

- interpretação estruturada;
- depende de inferência.

Dado, Conhecimento e Contexto

Dado

- informação isolada;
- matéria-prima bruta.

Exemplo

“220 horas”
registros, documentos...

Conhecimento

- interpretação estruturada;
- depende de inferência.

Exemplo

“O estágio obrigatório possui 220 horas.”

Contexto

- subconjunto relevante;
- limitado;
- pode estar incompleto;
- memória de trabalho do modelo.

Exemplo

“O curso exige estágio obrigatório de 220 horas realizado sob supervisão docente conforme regulamento institucional.”

Recuperação Vetorial é Aproximação

- Aproximação semântica
- Relevância provável
- Similaridade estatística

Problemas: ambiguidades, ruído, redundância, contexto envelhecido.

O que o LLM recebe em um pipeline RAG

Documentos → Texto → Chunks → Embeddings → Busca vetorial → Top- k → Prompt → LLM

Ideia central

Em um pipeline RAG, os chunks recuperados são a evidência externa disponibilizada ao LLM. O modelo ainda pode usar conhecimento aprendido no treinamento se o prompt não restringir a resposta.

Pergunta do usuário → Embedding da pergunta → Vector store → Contexto recuperado → Resposta

Bancos vetoriais

Modelo Tradicional de Dados

BANCO DE DADOS RELACIONAL TRADICIONAL (SQL)

Excelentes para dados estruturados, textos exatos e números.



Tabela: alunos

id	nome	idade	curso	nota
1	Ana	21	SI	9.2
2	Bruno	22	SI	8.7
3	Carla	20	CC	9.5
...	...	...	...	...



Tabelas
estruturadas



Textos exatos
(palavras-chave)



Números e
cálculos



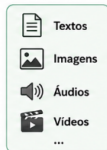
Resumo:

SQL é ideal quando os dados são estruturados, determinísticos e precisam de precisão exata, consistência e integridade.

BANCO DE DADOS VETORIAL

Projetado para armazenar, indexar e consultar dados como vetores (embeddings) e avaliar a similaridade semântica entre conceitos.

1. DADOS NÃO ESTRUTURADOS

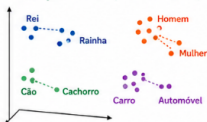


2. GERAÇÃO DE EMBEDDINGS

0.21	-0.73	0.11	...	0.85
-0.31	0.42	-0.22	...	0.33
0.58	-0.12	0.67	...	-0.21
-0.44	0.18	-0.91	...	0.12

...

3. ESPAÇO VETORIAL (ALTA DIMENSÃO)



Conceitos semanticamente próximos ficam próximos no espaço vetorial.

4. BUSCA POR SIMILARIDADE SEMÂNTICA

Q rei

Resultados (ordenados por similaridade):

- 1 Rainha 0.92
 - 2 Monarca 0.89
 - 3 Imperador 0.87
 - 4 Soberano 0.85
- ...



Encontra conceitos relacionados mesmo sem correspondência exata de palavras.



Em vez de buscar correspondências exatas, o banco vetorial entende o significado. Mais contexto. Mais relevância. Melhores respostas para IA.

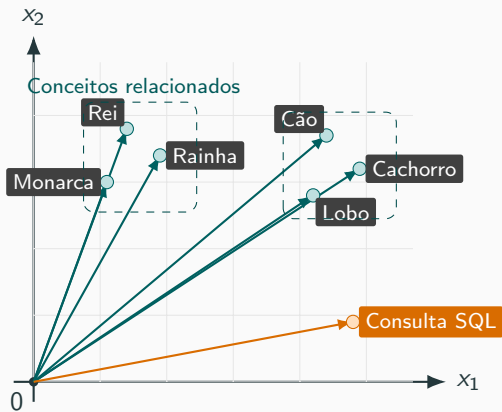


Embedding:

- representação vetorial do texto;
- aproximação matemática de significado;
- vetor numérico de alta dimensão.

textos semanticamente próximos
ficam próximos no espaço vetorial

Embeddings e Espaço Vetorial



Embedding

Representação numérica de um texto, imagem ou outro objeto.

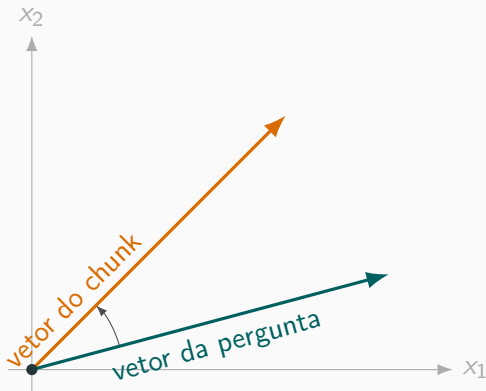
Espaço vetorial

Cada embedding ocupa uma posição em um espaço de muitas dimensões.

Proximidade semântica

Conceitos semanticamente relacionados tendem a ocupar regiões próximas.

Similaridade de Cosseno



A similaridade de cosseno compara a **direção** de dois vetores:

$$\text{sim}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

- próximo de 1: vetores semelhantes;
- próximo de 0: pouca relação;
- próximo de -1 : direções opostas.

Na recuperação

Quanto menor o ângulo entre a pergunta e o chunk, maior tende a ser sua similaridade semântica.

O que é um Banco de Dados Vetorial?

Um banco de dados vetorial é um sistema especializado para:

- armazenar embeddings;
- indexar vetores;
- realizar buscas por similaridade;
- recuperar vizinhos mais próximos.

Enquanto bancos relacionais respondem:

```
SELECT * FROM disciplinas
```

bancos vetoriais respondem:

“Quais vetores são mais parecidos com este?”

Banco Relacional vs. Banco Vetorial

	Banco relacional	Banco vetorial
Unidade armazenada	linhas e colunas	vetores e metadados
Busca principal	igualdade, filtros, intervalos, joins e agregações	proximidade entre vetores
Exemplo	<code>WHERE codigo = 'CI1218'</code>	“vetores mais próximos desta pergunta”
Índices	B-tree, hash, GiST, GIN	HNSW, IVF e estruturas ANN equivalentes
Resultado	geralmente exato e determinístico	pode ser exato ou aproximado
Uso típico	sistemas transacionais, relatórios, dados estruturados	RAG, recomendação, busca semântica e recuperação multimodal

PostgreSQL com pgvector ou banco vetorial dedicado?

PostgreSQL + pgvector	Banco vetorial dedicado
Integra dados relacionais e embeddings no mesmo banco	Especializado em armazenamento e busca vetorial em escala
Permite SQL, joins, transações, filtros e governança já existentes	Pode oferecer distribuição, replicação, APIs e otimizações próprias para vetores
Boa escolha quando os dados estruturados já estão no PostgreSQL	Boa escolha quando há grande volume, baixa latência ou arquitetura distribuída
Menor complexidade arquitetural	Maior especialização operacional

A questão não é “PostgreSQL ou vetores”. PostgreSQL também pode operar com vetores via pgvector; a escolha depende de escala, governança, latência e integração com dados estruturados.

Soluções em Memória vs. Persistentes

	Índice local / em memória <i>Ex.: FAISS local</i>	Banco vetorial persistente <i>Ex.: ChromaDB, Weaviate, Pinecone</i>
Custo	Baixo para protótipos e coleções pequenas. Utiliza os recursos da própria máquina.	Médio a alto, dependendo da infraestrutura, do volume armazenado e do serviço utilizado.
Latência	Muito baixa, pois a busca ocorre diretamente na memória ou no armazenamento local.	Baixa, mas pode incluir comunicação de rede, serialização e operações de entrada e saída.
Escalabilidade	Limitada principalmente pela memória, pelo disco e pela capacidade computacional da máquina.	Pode ser horizontalmente escalável, com replicação, particionamento e distribuição.
Persistência	O índice pode ser mantido somente em memória ou explicitamente salvo em disco.	Dados, índices e metadados são mantidos e recuperados entre diferentes execuções.
Governança	Controle de acesso, versionamento e atualização precisam ser implementados pela aplicação.	Pode oferecer RBAC, auditoria, metadados, replicação, backup e atualização controlada.
Operação	Mais simples no início, mas exige código próprio para atualização, sincronização e recuperação.	Maior complexidade operacional, mas com recursos prontos para ambientes de produção.

Trade-offs em Sistemas Reais

Objetivo	Consequência
Menor latência	Menor recall
Maior precisão	Maior custo
Chunks maiores	Mais ruído
Chunks menores	Menos contexto
Top-k maior	Mais redundância
Embeddings maiores	Mais memória

Pipeline

Problema Fundamental

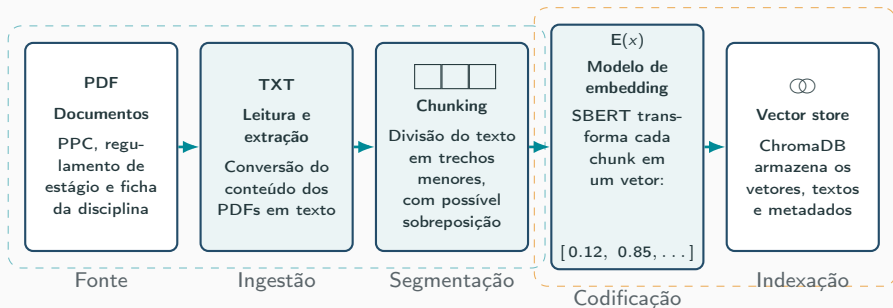
LLMs possuem:

- limite de contexto;
- custo por token;
- aumento de latência.

Conclusão:

recuperar contexto relevante

Pipeline — Fase 1: ingestão e indexação



O que pode falhar na ingestão?

PDF mal extraído, tabelas quebradas, caracteres ausentes, páginas escaneadas ou documentos duplicados.

O que pode falhar na indexação?

Chunks inadequados, embeddings pouco representativos, metadados ausentes ou índice desatualizado.

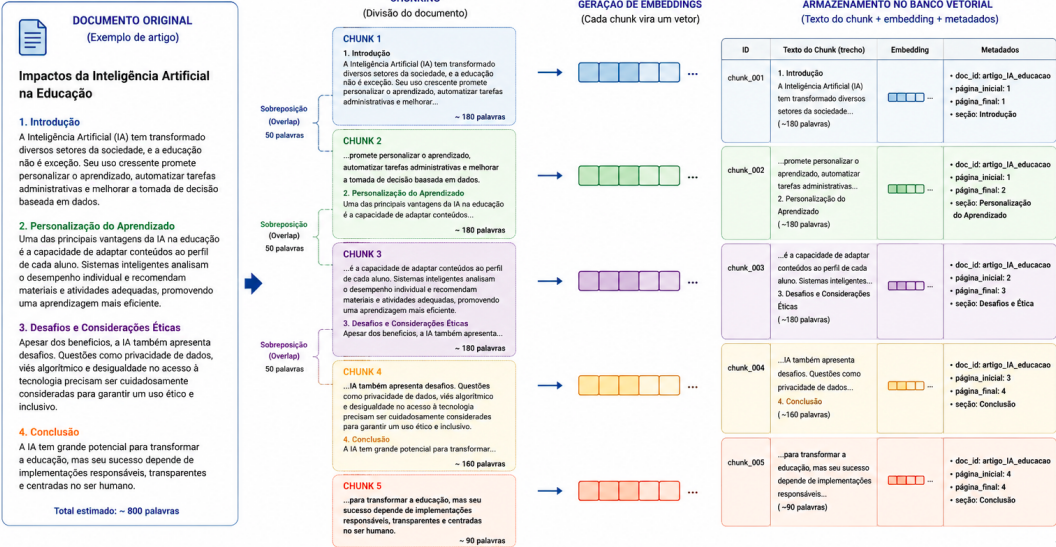
Chunking:

- divisão do documento em partes menores.

Problemas:

- chunks pequenos perdem contexto;
- chunks grandes geram ruído;
- overlap excessivo aumenta redundância.

Exemplo de chunking



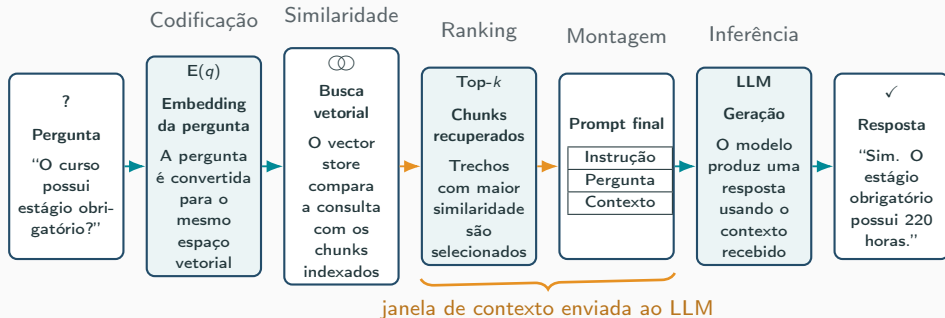
Chunking Simples

```
def gerar_chunks(  
    texto,  
    tamanho=800,  
    overlap=150  
):
```

Perguntas importantes:

- o chunk contém informação suficiente?
- houve corte de contexto?
- há redundância?
- múltiplos assuntos foram misturados?

Pipeline — Fase 2: recuperação e geração



Recuperação

O sistema busca os chunks mais próximos da pergunta. O resultado depende do embedding, do índice, do valor de k e do critério de similaridade.

O LLM não consulta o banco diretamente

O modelo recebe a pergunta, as instruções e os chunks selecionados. O prompt deve restringir a resposta às evidências recuperadas.

Busca Aproximada (ANN)

ANN:

Approximate Nearest Neighbors

Objetivo:

- acelerar busca vetorial;
- reduzir custo computacional.

Trade-off:

- maior velocidade;
- menor precisão.

Indexação Vetorial: Recuperação em Alta Velocidade

Problema

Buscar milhões de vetores comparando um a um é caro:

$$O(N)$$

Quanto maior a base, maior o tempo de busca.

Bancos vetoriais usam **ANN**:

Approximate Nearest Neighbor.

A busca deixa de comparar contra todos os vetores e passa a explorar apenas regiões promissoras.

Indexação Vetorial: Recuperação em Alta Velocidade

Problema

Buscar milhões de vetores comparando um a um é caro:

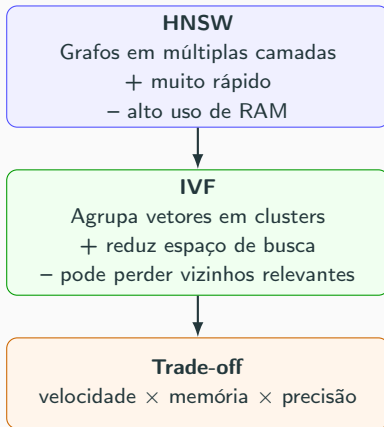
$$O(N)$$

Quanto maior a base, maior o tempo de busca.

Bancos vetoriais usam **ANN**:

Approximate Nearest Neighbor.

A busca deixa de comparar contra todos os vetores e passa a explorar apenas regiões promissoras.



Indexação Vetorial: Recuperação em Alta Velocidade

Problema

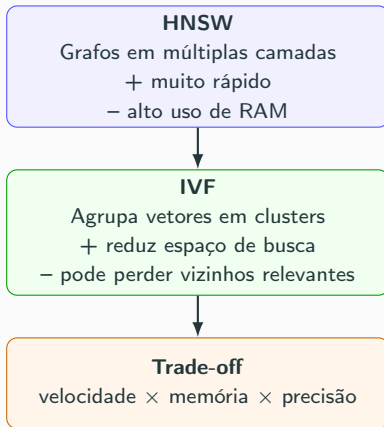
Buscar milhões de vetores comparando um a um é caro:

$$O(N)$$

Quanto maior a base, maior o tempo de busca.

Bancos vetoriais usam **ANN**:
Approximate Nearest Neighbor.

A busca deixa de comparar contra todos os vetores e passa a explorar apenas regiões promissoras.



ANN não garante sempre o vizinho matematicamente exato; ele busca uma resposta suficientemente próxima em tempo viável.

Escolhendo um Banco de Dados Vetorial

Critério de escolha

A decisão depende dos requisitos de infraestrutura, escala, governança e integração com sistemas existentes.

Tipo	Exemplos	Melhor uso
Nativos de IA / Dedicados Extensões Tradicionais	Pinecone, Milvus, Weaviate, Chroma pgvector, Redis, OpenSearch	Vetores de alta dimensão, indexação ANN rápida e aplicações de IA generativa em escala. Quando já existe infraestrutura relacional ou de busca e os vetores precisam ficar próximos dos dados existentes.
Data Lakes Especializados	Deep Lake, Zilliz Cloud	Pipelines complexos de ML, dados multimodais e armazenamento voltado a conjuntos de dados de deep learning.

Laboratório e avaliação da recuperação

Documentos utilizados

- PPC;
- regulamento de estágio;
- ficha da disciplina CI1218.

Implementaremos

- leitura dos PDFs;
- chunking;
- embeddings;
- ChromaDB;
- recuperação top- k .

Perguntas de teste

- Quantas horas possui o estágio obrigatório?
- A disciplina de Banco de Dados aborda otimização?
- Qual é a frequência mínima?

Gerando Embeddings

```
from sentence_transformers import SentenceTransformer

modelo = SentenceTransformer(
    "sentence-transformers/"
    "paraphrase-multilingual-MiniLM-L12-v2"
)

embedding = modelo.encode(
    "Banco_de_dados"
)

print(len(embedding)) # 384
```

Armazenamento Vetorial

Armazenamento vetorial:

- armazena embeddings;
- realiza busca vetorial;
- retorna chunks similares.

Importante:

similaridade não implica relevância

```
resultados = collection.query(  
    query_embeddings=[  
        embedding_pergunta  
    ],  
    n_results=3  
)
```

Top-k:

- quantidade de chunks recuperados.

Problemas:

- top-k pequeno: contexto insuficiente;
- top-k grande: ruído e redundância.

Exemplo de Recuperação

Pergunta:

“O curso possui estágio obrigatório?”

Rank	Chunk recuperado	Similaridade
1	estágio obrigatório 220h	0.89
2	atividades formativas	0.81
3	trabalho de conclusão	0.78

- similaridade garante relevância?
- por que chunks incorretos apareceram?
- o primeiro resultado contém evidência suficiente?

Como Avaliar os Resultados

Dimensão	O que observar
Relevância	O chunk contém a evidência necessária?
Ranking	A evidência aparece nas primeiras posições?
Redundância	Há trechos quase iguais entre os recuperados?
Cobertura	Diferentes aspectos da pergunta foram cobertos?
Tokens	Quanto contexto será enviado ao LLM?
Latência	Quanto tempo a busca levou?
Fonte	Documento, página e seção estão identificados?

Medindo Latência da Recuperação

```
from time import perf_counter

inicio = perf_counter()

resultados = buscar(
    collection,
    modelo,
    pergunta,
    k=3
)

latencia_ms = (perf_counter() - inicio) * 1000

print(f"Latncia: {latencia_ms:.2f} ms")
```

Mais Contexto Nem Sempre Ajuda

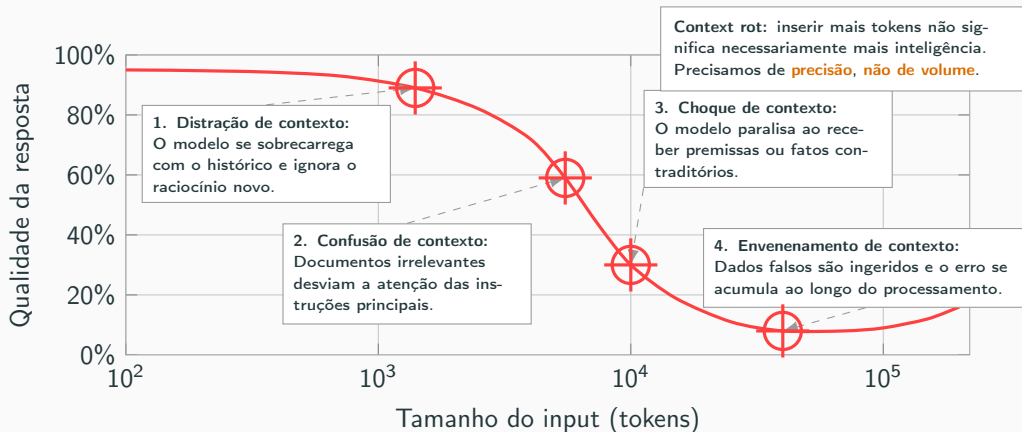
Problemas causados por excesso de contexto:

- redundância;
- competição entre chunks;
- atenção difusa;
- aumento de latência;
- aumento de custo.

Exemplo:

- $\text{top-k} = 4 \rightarrow$ resposta correta;
- $\text{top-k} = 20 \rightarrow$ resposta pior.

Riscos do Excesso de Contexto



Curva hipotética e conceitual: a forma exata depende do modelo, da tarefa e da qualidade do contexto.

Contexto Problemático

Pergunta:

“Qual é a frequência mínima?”

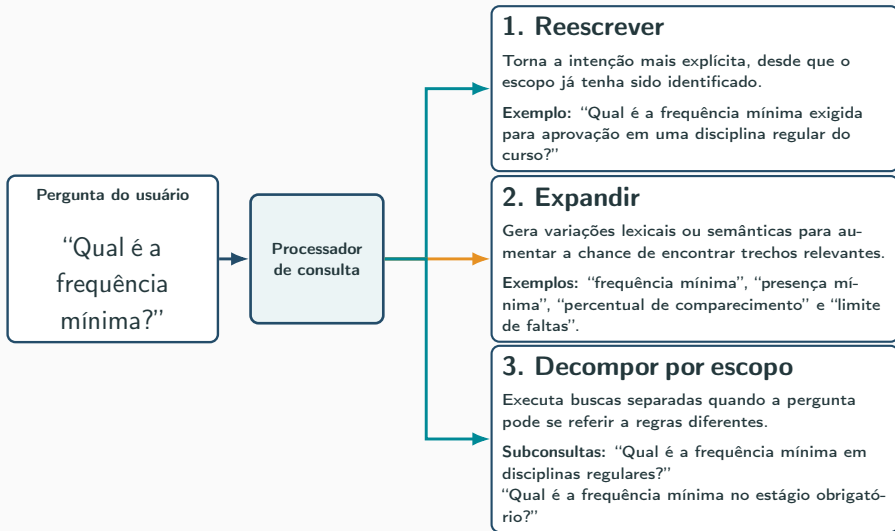
Possíveis interpretações:

- frequência da disciplina;
- frequência do estágio;
- frequência para aprovação.

Problema:

- recuperação ambígua;
- contexto insuficiente;
- chunks conflitantes.

Transformação da consulta antes da recuperação

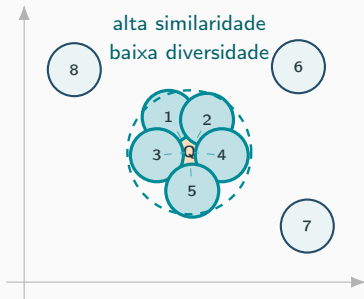


Redundância e refinamento

Similaridade ou diversidade?

Apenas similaridade

Os chunks mais próximos da pergunta são selecionados, mesmo que tragam informações repetidas.



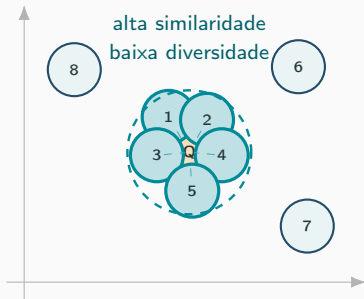
Problema

O top-k pode retornar vários trechos do mesmo artigo ou documento. O contexto fica redundante e cobre poucos aspectos da pergunta.

Similaridade ou diversidade?

Apenas similaridade

Os chunks mais próximos da pergunta são selecionados, mesmo que tragam informações repetidas.

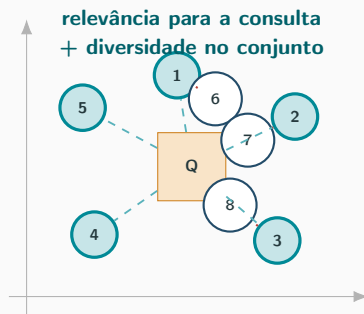


Problema

O top-k pode retornar vários trechos do mesmo artigo ou documento. O contexto fica redundante e cobre poucos aspectos da pergunta.

Maximal Marginal Relevance

Cada nova seleção considera a relevância para a pergunta e a diferença em relação aos chunks já escolhidos.



Resultado

O contexto tende a apresentar trechos relevantes de diferentes seções, documentos ou perspectivas,

Como o MMR seleciona os chunks?

A cada passo, o MMR escolhe o documento d que maximiza:

$$\text{MMR}(d) = \lambda \text{sim}(d, q) - (1 - \lambda) \max_{d' \in S} \text{sim}(d, d')$$

q : embedding da pergunta;

S : conjunto de chunks já selecionados;

$\text{sim}(d, q)$: relevância do chunk para a pergunta;

$\text{sim}(d, d')$: redundância em relação aos selecionados;

λ : equilíbrio entre relevância e diversidade.

λ próximo de 1

Prioriza relevância. O comportamento se aproxima do ranking tradicional por similaridade.

λ menor

Aumenta a diversidade, mas pode incluir chunks menos diretamente relacionados à pergunta.

Diagnóstico e limites

Diagnóstico de Falhas: ingestão e indexação

! Sintoma

A pergunta é pertinente, mas o documento ou o trecho esperado **não aparece nos resultados**.

Exemplo:

A pergunta “Quantas horas possui o estágio obrigatório?” não recupera o artigo que informa as **220 horas**.

Pergunta de diagnóstico

A informação existe no PDF e foi corretamente extraída?

Q Causas prováveis

- texto do PDF extraído incorretamente;
- página escaneada ou tabela não reconhecida;
- artigo dividido entre chunks;
- chunk grande demais, diluindo o conteúdo;
- documento não indexado;
- índice antigo ou duplicado;
- metadados ausentes ou incorretos;
- modelo de embedding inadequado ao idioma ou domínio.

Correções

1. inspecionar o texto extraído;
2. verificar se o trecho existe nos chunks;
3. dividir por artigos, parágrafos ou seções;
4. ajustar tamanho e sobreposição;
5. incluir metadados de documento, página e seção;
6. remover duplicatas e reconstruir o índice;
7. testar o embedding com consultas conhecidas.

Diagnóstico de Falhas: busca, contexto e geração

Sintoma

O sistema recupera trechos relacionados, mas:

- retorna chunks redundantes;
- mistura regras de escopos diferentes;
- omite a evidência principal;
- produz uma resposta não sustentada pelo contexto.

Exemplo: para “Qual é a frequência mínima?”, mistura frequência em disciplinas e frequência no estágio.

Causas prováveis

- consulta ambígua;
- valor de k excessivo ou insuficiente;
- chunks redundantes ou contraditórios;
- ausência de filtros por metadados;
- prompt sem restrição de evidência;
- LLM infere além do contexto recuperado.

Correções

- esclarecer, reescrever ou decompor a consulta;

Problema:

- documentos mudam;
- embeddings permanecem;
- chunks podem ficar inconsistentes.

Exemplo:

- PPC antigo;
- regulamento atualizado.

Consequência

A recuperação pode estar correta, mas a resposta ainda ser errada.

Ingestão

- PDFs ruins;
- OCR;
- texto quebrado.

Indexação

- chunking ruim;
- embeddings inadequados.

Busca

- top-k inadequado;
- ranking incorreto.

Quando a recuperação falhar:

1. o PDF foi lido corretamente?
2. o chunk ficou adequado?
3. o embedding representa bem o texto?
4. o top-k foi suficiente?
5. houve redundância?
6. o contexto enviado ao LLM faz sentido?

Vetor Não Resolve Tudo

Pergunta:

“Qual disciplina possui maior carga horária?”

Busca vetorial:

- ruim para agregações;
- ruim para ordenações;
- ruim para cálculos.

SQL:

- estruturado;
- preciso;
- determinístico.

Recuperação não é Compreensão

Embeddings:

- aproximam significado;
- não verificam verdade;
- não entendem domínio;
- não possuem raciocínio simbólico.

Conclusão

Similaridade semântica não implica compreensão.

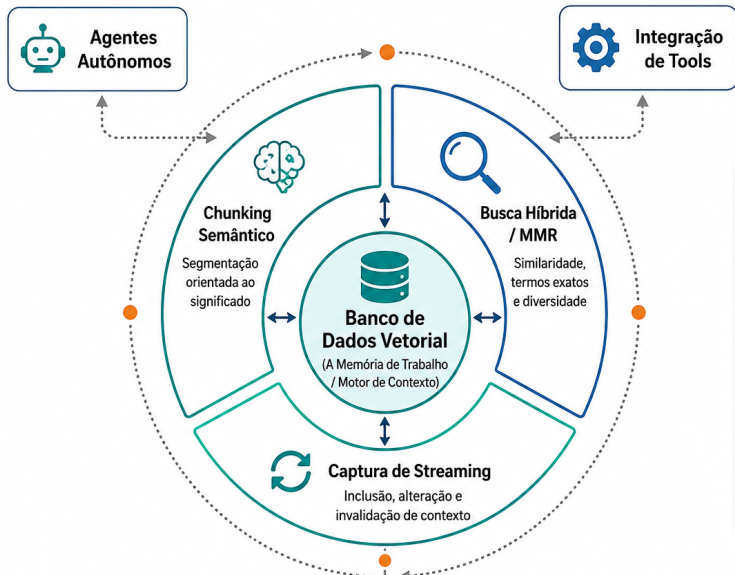
Pergunta importante:

“O sistema recuperou algo parecido
ou realmente relevante?”

Recuperação vetorial é:

- probabilística;
- aproximada;
- sujeita a ruído;
- dependente da indexação.

O Banco Vetorial no Ecossistema de Contexto



O banco vetorial não é apenas um local de armazenamento; é o **substrato cognitivo** da Inteligência Artificial moderna.

Engenharia de Contexto

é a arte de **gerenciar**, **refinar** e **orquestrar** esse substrato em tempo real.

- recuperação vetorial é aproximação;
- embeddings possuem limitações;
- contexto é recurso escasso;
- mais contexto pode piorar respostas;
- RAG depende da qualidade da recuperação.

Próximos tópicos:

- BM25;
- busca híbrida;
- reranking;
- filtros;
- refinamento;
- compressão de contexto.